📘 CSS — COMPLETE NOTES

Beginner → Advanced → Professional

🧭 CSS ROADMAP

```
LEVEL 1 — 🟢 Beginner
CSS Fundamentals
        ↓
LEVEL 2 — 🟢 Beginner+
Layout
        ↓
LEVEL 3 — 🟡 Intermediate
Responsive Design
        ↓
LEVEL 4 — 🟠 Advanced
Professional UI Styling
        ↓
LEVEL 5 — 🔴 Expert
Advanced CSS + Accessibility + Performance
```

LEVEL 1 — 🟢 BEGINNER

CSS Fundamentals

1. What is CSS?

CSS = Cascading Style Sheets

HTML:

```
What exists?
```

CSS:

```
How does it look?
```

Example:

```
<h1>Hello World</h1>
h1 {
    color: blue;
    font-size: 40px;
}
```

Browser output

```
Hello World
```

The heading appears **blue and large**.

---

2. CSS Syntax

```
selector {
    property: value;
}
```

Example:

```
p {
    color: red;
    font-size: 20px;
}
```

Part	Meaning
`p`	selector
`color`	property
`red`	value
`;`	ends declaration

---

3. Three ways to add CSS

A. Inline CSS

```
<h1 style="color: red;">Hello</h1>
```

Output:

```
Hello
```

🔴 Red heading.

Use?

Small experiments.

Professional?

Generally avoid for normal application styling.

B. Internal CSS

```
<style>
    h1 {
        color: blue;
    }
</style>
```

Good for:

Small single-page examples

C. External CSS ⭐

HTML:

```
<link rel="stylesheet" href="style.css">
```

CSS:

```
h1 {
    color: blue;
}
```

Professional projects normally favor organized external/component styles rather than scattering inline styles everywhere.

---

4. CSS Comments

```
/* This is a comment */

h1 {
    color: blue;
}
```

Browser:

```
Comment is NOT displayed.
```

---

5. Selectors

Element selector

```
p {
    color: red;
```

```
}
```

All <p> elements become red.

Class selector ⭐

HTML:

```
<p class="warning">Wrong password</p>
```

CSS:

```
.warning {
    color: red;
}
```

Output:

```
Wrong password
```

🔴 Red text.

Classes are heavily used in real applications.

ID selector

```
<h1 id="title">Dashboard</h1>
#title {
    color: blue;
}
```

Use IDs carefully for styling; classes are generally more reusable.

Universal selector

```
* {
    box-sizing: border-box;
}
```

Selects all elements.

---

6. Group selectors

Instead of:

```
h1 {
    color: blue;
}

h2 {
    color: blue;
}

p {
    color: blue;
}
```

write:

```
h1,
h2,
p {
    color: blue;
}
```

---

7. Descendant selector

```
.card p {
    color: gray;
}
```

Means:

Select `p` elements inside `.card`.

---

8. Child selector

```
.card > p {
    color: red;
}
```

Means:

Select direct child `<p>` elements.

---

9. Attribute selector

```
input[type="email"] {
    border: 2px solid blue;
}
```

Only email inputs receive the style.

---

10. Colors

Named color

```
color: red;
```

HEX

```
color: #ff0000;
```

RGB

```
color: rgb(255, 0, 0);
```

RGBA

```
color: rgba(255, 0, 0, 0.5);
```

Modern color functions

You may also encounter:

```
hsl()
hsla()
oklch()
color()
```

Don't memorize every color system immediately.

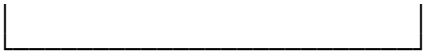
---

11. Background

```
body {
    background-color: lightgray;
}
```

Output:

```
┌──────────────────────────┐
│   entire page = gray     │
```



---

12. Background image

```
.hero {
    background-image: url("hero.jpg");
    background-size: cover;
    background-position: center;
}
```

Common:

```
background-size: cover
background-size: contain
background-position
background-repeat
```

---

13. Text styling

```
h1 {
    color: #111827;
    font-size: 40px;
    font-weight: 700;
    text-align: center;
}
```

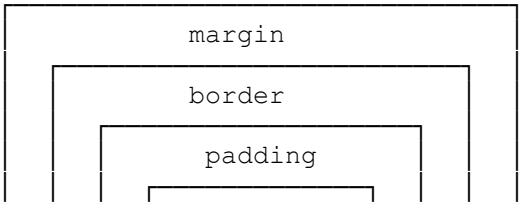
Controls:

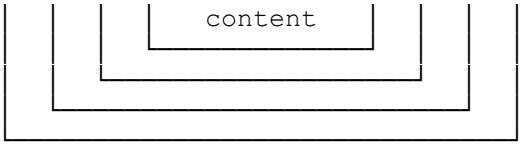
color
font-size
font-family
font-weight
font-style
text-align
text-decoration
line-height
letter-spacing

---

14. Box Model ⭐ ⭐ ⭐

Every element is basically treated as a box:





Remember:

```
Content
 ↓
Padding
 ↓
Border
 ↓
Margin
```

---

15. Padding

```
.card {
    padding: 20px;
}
```

Adds space **inside** the card.

---

16. Margin

```
.card {
    margin: 20px;
}
```

Adds space **outside** the card.

Easy memory

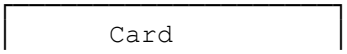
```
padding = inside
margin  = outside
```

---

17. Border

```
.card {
    border: 1px solid gray;
}
```

Output:



---

18. box-sizing

Very important:

```
* {
    box-sizing: border-box;
}
```

It makes sizing easier to reason about because declared width/height includes padding and border.

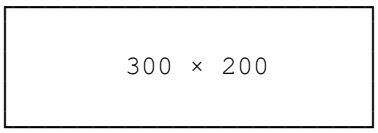
This is a common professional baseline.

---

19. Width and Height

```
.box {
    width: 300px;
    height: 200px;
}
```

Output:



But don't make everything rigid.

Modern applications frequently use:

```
width
max-width
min-width
min-height
```

together.

---

20. display

Important values:

```
block
inline
inline-block
none
flex
grid
```

Example:

```
.box {
    display: none;
}
```

Output:

```
Nothing appears.
```

---

21. `overflow`

```
.box {
    overflow: auto;
}
```

Useful when content is larger than the box.

Common:

```
visible
hidden
auto
scroll
```

---

22. CSS units

Unit	Meaning
`px`	CSS pixel
`%`	Relative size
`rem`	Root font-size based
`em`	Current/contextual font-size based
`vw`	Viewport width
`vh`	Viewport height
`fr`	Grid fraction

For application UI, become especially comfortable with:

```
px
%
rem
fr
clamp()
```

LEVEL 2 — 🟢 BEGINNER+

Layout

This is where CSS becomes much more powerful.

---

23. Flexbox ⭐⭐⭐

Use Flexbox when you primarily need to arrange things along **one dimension**.

```
.container {
    display: flex;
}
```

Example:

```
<div class="container">
    <button>Login</button>
    <button>Signup</button>
</div>
.container {
    display: flex;
    gap: 10px;
}
```

Output:

[Login] [Signup]

---

24. `flex-direction`

```
.container {
    display: flex;
    flex-direction: column;
}
```

Output:

```
[ Login ]

[ Signup ]
```

Values:

```
row
column
row-reverse
column-reverse
```

---

25. `justify-content`

Controls distribution along the **main axis**.

```
.container {
    display: flex;
    justify-content: center;
}
```

Output:

```
        [ Login ] [ Signup ]
```

Common:

```
flex-start
center
flex-end
space-between
space-around
space-evenly
```

---

26. `align-items`

Controls alignment on the **cross axis**.

```
.container {
    display: flex;
    align-items: center;
}
```

Very commonly used for:

```
Navbar
Buttons
Cards
Toolbar
```

---

27. `gap`

Instead of adding margins everywhere:

```
.container {
```

```
    display: flex;
    gap: 20px;
}
```

Output:

[Button] [Button] [Button]

---

28. Flex wrapping

```
.container {
    display: flex;
    flex-wrap: wrap;
}
```

If there isn't enough space:

```
[1] [2] [3]
[4] [5]
```

rather than forcing everything onto one line.

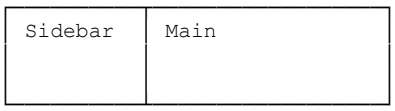
---

29. CSS Grid ⭐ ⭐ ⭐

Use Grid when you need **two-dimensional layout**.

```
.dashboard {
    display: grid;
    grid-template-columns: 250px 1fr;
}
```

Output:

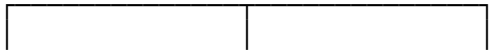


---

30. Grid columns

```
grid-template-columns: 1fr 1fr;
```

Output:



```
┌──────────────┬──────────────┐
│     50%      │     50%      │
└──────────────┴──────────────┘
```

---

31. `repeat()`

Instead of:

```
grid-template-columns: 1fr 1fr 1fr;
```

write:

```
grid-template-columns: repeat(3, 1fr);
```

---

32. `minmax()`

```
grid-template-columns:
    repeat(auto-fit, minmax(250px, 1fr));
```

Excellent for responsive card grids.

Output approximately:

`Desktop:`

[Card] [Card] [Card]

`Mobile:`

[Card]
[Card]
[Card]

---

33. Grid rows

```
grid-template-rows: 100px 1fr;
```

---

34. Grid gap

```
grid-gap: 20px;
```

Modern CSS commonly uses:

```
gap: 20px;
```

35. Positioning

static

Normal document flow.

relative

```
.box {
    position: relative;
}
```

Keeps its normal space while allowing offsets and positioning context for descendants.

---

absolute

```
.badge {
    position: absolute;
    top: 10px;
    right: 10px;
}
```

Useful for:

```
Notification badge
Close button
Floating icon
Overlay
```

---

fixed

```
.help {
    position: fixed;
    right: 20px;
    bottom: 20px;
}
```

The element is positioned relative to the viewport.

Example:

```
                              ?
                      fixed button
```

---

sticky

```
.sidebar {
```

```
    position: sticky;
    top: 20px;
}
```

Useful for:

```
Sticky navbar
Sticky sidebar
```

LEVEL 3 — 🟡 INTERMEDIATE

Responsive Design

---

36. Responsive design

Goal:

```
Desktop
   ↓
Tablet
   ↓
Mobile
```

Same application.

Different layout.

---

37. `max-width`

Instead of:

```
.container {
    width: 1200px;
}
```

prefer:

```
.container {
    width: 100%;
    max-width: 1200px;
    margin-inline: auto;
}
```

Output:

```
Small screen → fills available width

Large screen → stops at 1200px
```

---

38. Media queries ⭐ ⭐ ⭐

```
@media (max-width: 768px) {
    .dashboard {
        grid-template-columns: 1fr;
    }
}
```

Desktop:

```
Sidebar | Main
```

Mobile:

```
Sidebar
Main
```

---

39. Mobile-first

Base:

```
.dashboard {
    grid-template-columns: 1fr;
}
```

Then:

```
@media (min-width: 768px) {
    .dashboard {
        grid-template-columns: 250px 1fr;
    }
}
```

Mental model:

```
Mobile first
    ↓
enhance for larger screens
```

---

40. rem

If:

```
html {
```

```
    font-size: 16px;
}
```

then:

```
font-size: 2rem;
```

is typically:

```
32px
```

Good for scalable typography and spacing.

---

41. `clamp()` ⭐⭐⭐

```
h1 {
    font-size: clamp(2rem, 5vw, 4rem);
}
```

Meaning:

```
minimum = 2rem
preferred = 5vw
maximum = 4rem
```

So text grows smoothly rather than changing only at breakpoints.

---

42. `min()` and `max()`

```
width: min(90%, 1200px);
```

The width will not exceed the chosen maximum calculation.

And:

```
padding: max(16px, 3vw);
```

ensures padding doesn't drop below the minimum.

---

43. Responsive navigation

Desktop:

Logo Home Features Pricing Login

Mobile:

Logo ☰

CSS can change visibility/layout:

```
.desktop-menu {
    display: flex;
}

.mobile-menu {
    display: none;
}

@media (max-width: 768px) {
    .desktop-menu {
        display: none;
    }

    .mobile-menu {
        display: block;
    }
}
```

JavaScript will later make the menu **actually open and close**.

LEVEL 4 — 🔴 ADVANCED

Professional UI Styling

---

44. CSS variables ⭐ ⭐ ⭐

```
:root {
    --primary: #2563eb;
    --text: #111827;
    --background: #ffffff;
}
```

Use:

```
button {
    background: var(--primary);
}

body {
```

```
    color: var(--text);
    background: var(--background);
}
```

Why?

Change one variable:

```
Blue → Purple
```

and many components can update consistently.

---

45. Design tokens

Organize common values:

```
:root {
    --color-primary: #2563eb;

    --space-sm: 8px;
    --space-md: 16px;
    --space-lg: 24px;

    --radius-sm: 6px;
    --radius-md: 10px;
}
```

This gives your UI consistency.

---

46. Pseudo-classes

```
:hover
:focus
:focus-visible
:active
:disabled
:checked
:first-child
:last-child
:nth-child()
```

Example:

```
button:hover {
    transform: translateY(-2px);
}
```

Output:

```
Normal:

[ Generate ]

Hover:

[ Generate ] ↑
```

---

47. `:focus-visible`

```
button:focus-visible {
    outline: 3px solid blue;
}
```

Important for keyboard accessibility.

---

48. Pseudo-elements

```
.title::before {
    content: "✨ ";
}
```

Output:

```
✨ AI Email Writer
```

Common:

```
::before
::after
```

Useful for visual decoration and indicators.

---

49. Transitions

Without:

```
button:hover {
    transform: translateY(-2px);
}
```

The change may feel immediate.

With:

```
button {
    transition: transform 0.2s ease;
}
```

The movement becomes smooth.

---

50. Transform

```
.card:hover {
    transform: scale(1.02);
}
```

Common:

```
translate
scale
rotate
```

---

51. Animations

```
@keyframes pulse {
    from {
        transform: scale(1);
    }

    to {
        transform: scale(1.05);
    }
}

.icon {
    animation: pulse 1s infinite alternate;
}
```

Output:

```
Icon
↓
slightly grows
↓
shrinks
↓
grows
...
```

Don't animate everything.

---

52. Forms

Example:

```
input,
textarea {
    width: 100%;
    padding: 12px;
    border: 1px solid #ccc;
    border-radius: 8px;
}
```

Professional forms should consider:

```
Normal
Hover
Focus
Error
Disabled
Loading
Success
```

---

53. Professional button states

```
button:hover {
    opacity: 0.9;
}

button:active {
    transform: scale(0.98);
}

button:disabled {
    opacity: 0.5;
    cursor: not-allowed;
}
```

---

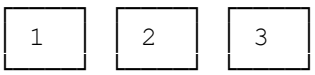
54. Responsive card system

```
.cards {
    display: grid;
    grid-template-columns:
        repeat(auto-fit, minmax(250px, 1fr));

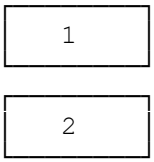
    gap: 24px;
}
```

Browser behavior:

```
Wide screen:
```



Narrow screen:



LEVEL 5 — 🔴 EXPERT

Advanced CSS + Production Thinking

---

55. CSS architecture

Think in reusable components:

```
Navbar
Button
Card
Input
Modal
Sidebar
Badge
Alert
Table
Form
```

Each component should have:

```
Structure
 ↓
Layout
 ↓
Responsive behavior
 ↓
States
 ↓
Accessibility
```

This thinking transfers directly to React.

---

56. Cascade

CSS means:

Cascading Style Sheets

When multiple rules apply, the browser determines which wins based on things such as:

```
Origin
Importance
Layers
Specificity
Source order
```

Don't solve every conflict by adding more selectors.

---

57. Specificity

Generally, selectors with greater specificity have more weight.

For example:

```
p {
    color: blue;
}

.card p {
    color: red;
}
```

Inside `.card`:

```
red wins
```

because `.card p` is more specific.

---

58. Cascade layers

Modern CSS:

```
@layer reset, components, utilities;
```

Useful in larger projects where you need predictable control over competing style rules.

---

59. CSS nesting

Modern CSS supports nesting:

```
.card {
    padding: 20px;

    & h2 {
        font-size: 24px;
    }

    & p {
        color: gray;
    }
}
```

Useful, but avoid excessive nesting.

---

60. Logical properties

Instead of always thinking:

```
margin-left
margin-right
```

modern CSS can use:

```
margin-inline-start
margin-inline-end
margin-block-start
margin-block-end
```

This works better with different writing directions.

---

61. `aspect-ratio`

```
.video {
    aspect-ratio: 16 / 9;
}
```

Output:



Useful for:

Videos
Images
Cards
Preview areas

---

62. object-fit

```
img {
    width: 100%;
    height: 100%;
    object-fit: cover;
}
```

cover

Fills the box, potentially cropping.

contain

Shows the entire image, potentially leaving empty space.

---

63. Container queries ⭐ ⭐ ⭐

Instead of:

"How wide is the browser?"

a component can ask:

"How much space does my container have?"

Example:

```
.card-container {
    container-type: inline-size;
}

@container (min-width: 500px) {
    .card {
        /* wider layout */
    }
}
```

Very useful for reusable components.

---

64. Stacking context

`z-index` isn't simply:

```
999999 > 100
```

in every situation.

Different stacking contexts can affect which element appears above another.

Important concepts:

```
position
z-index
opacity
transform
isolation
```

---

65. `isolation`

```
.container {
    isolation: isolate;
}
```

Can help create a predictable stacking context for complex UI layering.

---

66. CSS performance

Don't blindly optimize everything.

Understand that expensive or unnecessary work can come from things such as:

```
Excessive animations
Complex effects
Unnecessary layout changes
Huge DOM trees
Heavy filters
```

For animations, `transform` and `opacity` are often preferable to repeatedly changing layout properties.

---

67. `prefers-reduced-motion`

```
@media (prefers-reduced-motion: reduce) {
    * {
        animation-duration: 0.01ms;
        transition-duration: 0.01ms;
    }
}
```

Allows the interface to respect users who prefer reduced motion.

---

68. Accessibility

Professional CSS must consider:

```
Keyboard users
Screen-reader users
Color contrast
Focus visibility
Reduced motion
Touch targets
Readable typography
```

Remember:

Beautiful ≠ professional if users cannot properly use it.

---

69. CSS DevTools ⭐ ⭐ ⭐

Learn to use:

```
Inspect
 ↓
Elements
 ↓
Styles
 ↓
Computed
 ↓
Layout
```

You should be able to answer:

```
Why isn't this centered?
Why is this width wrong?
Why isn't z-index working?
Which rule overrides this?
Why is the element overflowing?
```

This is more valuable than memorizing hundreds of properties.

🎯 CSS QUICK REFERENCE TABLE

Problem	CSS
Text color	`color`
Background	`background`
Space inside	`padding`
Space outside	`margin`
Border	`border`
Width limit	`max-width`
Flex layout	`display: flex`
Grid layout	`display: grid`
Space between items	`gap`
Center flex items	`justify-content`, `align-items`
Hide element	`display: none`
Floating element	`position: absolute`
Fixed to viewport	`position: fixed`
Sticky element	`position: sticky`
Responsive CSS	`@media`
Reusable values	`var()`
Smooth change	`transition`
Movement	`transform`
Animation	`@keyframes`
Fluid text	`clamp()`
Responsive grid	`minmax()`
Image cropping	`object-fit`
Fixed aspect shape	`aspect-ratio`
Component responsiveness	`@container`

🎯 WHAT EACH CSS LEVEL GIVES YOU

Level	Skill	You can build
🟢 **1**	Fundamentals	Styled webpages
🟢 **2**	Layout	Cards, navbars, dashboards
🟡 **3**	Responsive	Mobile + desktop websites
🟠 **4**	Professional UI	SaaS-style interfaces
🔴 **5**	Advanced CSS	Production-quality UI systems

💼 INTERVIEW / EXAM QUESTIONS

1. What is CSS and why is it called "cascading"?
2. Difference between class and ID?
3. Explain the CSS box model.
4. Difference between margin and padding?
5. What does `box-sizing: border-box` do?
6. Difference between Flexbox and Grid?
7. What are the main axes in Flexbox?
8. Difference between `justify-content` and `align-items`?
9. Difference between `relative`, `absolute`, `fixed`, and `sticky`?
10. What is responsive design?
11. What is mobile-first development?
12. What is a media query?
13. Why use `rem` instead of only `px`?
14. What does `clamp()` do?
15. What are CSS variables?
16. What is CSS specificity?
17. What is the cascade?
18. What are pseudo-classes?
19. Difference between `:focus` and `:focus-visible`?
20. What are pseudo-elements?
21. What is a stacking context?
22. Why might `z-index` not work as expected?
23. What are container queries?
24. What is `object-fit: cover`?
25. What is `aspect-ratio`?
26. How do you make a CSS layout accessible?
27. How would you make a dashboard responsive?
28. How would you structure CSS for a large React application?
29. How can CSS affect frontend performance?
30. How would you debug a CSS layout that is not behaving correctly?